



NORTHWESTERN UNIVERSITY

Computer Science Department

Technical Report
Number: NU-CS-2026-27

July, 2024

Theoretical Bounds and Structure of Byte-pair Encoding

Avi Shein

Abstract

Byte Pair Encoding (BPE) is a widely used greedy algorithm for constructing subword tokenizations in natural language processing, yet its theoretical guarantees remain only partially understood. Building on the framework introduced by László Kozma and Johannes Voderholzer, we study BPE as an optimization problem in which, given a string and a fixed budget of merge operations, the goal is to maximize the total utility of merges.

We first establish an upper bound showing that BPE achieves at most a $(1/2 + \varepsilon)$ approximation for any $\varepsilon > 0$, via an explicit family of hard instances. We then prove a lower bound of $(1 - e^{-2})/2 \approx 0.43$ under the assumption that the input consists of length-4 substrings separated by distinct delimiter symbols, providing a constant-factor guarantee in this setting.

Finally, we develop a conjectural analysis of BPE on strings drawn uniformly at random through a random maximum-edge model, supported by experiments. This model predicts near-optimal performance in expectation for the first $n^2/4$ merge steps, where n is the alphabet size. Together, these results refine existing bounds and offer new insight into the behavior of BPE

Keywords

Byte Pair Encoding, compression, tokenization, greedy algorithms, approximation algorithms, merge operations, theoretical guarantees, random strings

THEORETICAL BOUNDS AND STRUCTURE OF BYTE-PAIR ENCODING

A THESIS

Submitted to the Graduate School
in Partial Fulfillment of the Requirements
for the Degree

Master of Science

Field of Computer Science

by

Avi Shein
Northwestern University
Evanston, Illinois
June 2026

Abstract

Byte Pair Encoding (BPE) is a widely used greedy algorithm for constructing subword tokenizations in natural language processing, yet its theoretical guarantees remain only partially understood. Building on the framework introduced by László Kozma and Johannes Voderholzer, we study BPE as an optimization problem in which, given a string and a fixed budget of merge operations, the goal is to maximize the total utility of merges.

We first establish an upper bound showing that BPE achieves at most a $(\frac{1}{2} + \varepsilon)$ approximation for any $(\varepsilon > 0)$, via an explicit family of hard instances. We then prove a lower bound of $(\frac{1-e^{-2}}{2} \approx 0.43)$ under the assumption that the input consists of length-4 substrings separated by distinct delimiter symbols, providing a constant-factor guarantee in this setting.

Finally, we develop a conjectural analysis of BPE on strings drawn uniformly at random through a random maximum-edge model, supported by experiments. This model predicts near-optimal performance in expectation for the first $\frac{n^2}{4}$ merge steps, where n is the alphabet size. Together, these results refine existing bounds and offer new insight into the behavior of BPE.

Acknowledgments

Firstly, I am extremely grateful to my advisor, Konstantin Makarychev, for the time, effort, and guidance he has given me throughout my development as a master's student at Northwestern. Through both his algorithms courses and our weekly thesis meetings, I owe much of my growth as a student and researcher in theoretical computer science to Konstantin. I am excited to continue using the skills I have gained from this experience as I move forward into my PhD.

I am also incredibly thankful that I did not have to go through this process alone. I had the opportunity to work with Timofei Asinskii, who spent many hours with me working through proofs and ideas in the development of this thesis. He has been an excellent research partner and made the entire process, including our research meetings, something I looked forward to each week.

I am also grateful to my committee member, Aravindan Vijayaraghavan, for his time, feedback, and willingness to serve on my thesis committee.

Finally, I would like to thank my friends from Northwestern and from my undergraduate studies at Tufts for humoring my many ramblings as I worked through the proofs and ideas in this project. An extra special thanks goes to Trevor Sullivan and Liam Strand for taking on more than their fair share of these ramblings, especially as my thesis drew to a close and most of my thoughts seemed to center around the analysis of this algorithm. I am grateful to them for being patient sounding boards, continuing to put up with me, and making the process far more enjoyable.

Contents

Abstract	i
Acknowledgments	ii
1 Introduction	1
1.1 Motivation	1
1.2 Preliminaries	2
1.3 Problem Definition	3
1.4 Related Work	4
1.5 Main Results	5
2 Upper Bound on the BPE Approximation Factor	7
3 Lower Bound on BPE Approximation Factor	10
3.1 First m_1 steps	11
3.2 Latter m_2 steps	16
4 Tokenizing a Uniformly Random String	23
4.1 Optimal Strategy	23
4.2 Graph Structure of BPE	24
4.3 Imbalance of the Bipartition	26
4.4 Conjectural Models	26

4.5	Consequences of the Conjectures	27
4.6	Approximation Guarantees	30
5	Experimental Results	31
5.1	Setup	31
5.2	Results	32
5.3	Discussion	33
6	Conclusion and open questions	36
	Bibliography	39
A	Analysis of the Random Maximum-Edge Model	40

Chapter 1

Introduction

1.1 Motivation

Byte Pair Encoding (BPE), originally introduced for data compression [1] and later adapted for subword tokenization in natural language processing (NLP) [3], is a foundational preprocessing technique in modern natural language processing systems, particularly in large language models (LLMs). By greedily merging frequent adjacent symbol pairs, BPE defines the subword vocabulary that determines how all text is represented and processed. As a result, its behavior has a direct impact on model efficiency, context length, and training dynamics.

Despite its simplicity and ubiquity, BPE is surprisingly poorly understood from a theoretical perspective. The algorithm is inherently greedy, yet it operates over a highly non-local objective: each merge decision can significantly affect the utility of many future merges. This creates complex dependencies between merge steps, making it difficult to reason about how close BPE comes to an optimal merge sequence.

Recent work has begun to study BPE from an approximation perspective, showing that its worst-case performance lies between $\frac{1}{3}$ and $\frac{5}{8}$ of optimal [2]. This gap highlights a fundamental lack of understanding as it remains unclear how bad BPE

can be, and what structural properties of the input govern its performance.

In this work, we significantly narrow this gap and provide structural insight into when BPE performs well or poorly. We construct a family of instances showing that BPE achieves at most a $(\frac{1}{2} + \varepsilon)$ -approximation, and provide complementary guarantees in structured settings. We also analyze BPE on uniformly random strings, identifying regimes in which the algorithm performs near-optimally.

Beyond improving bounds, our goal is to understand why these extremes arise. Our analysis suggests that the key factor governing BPE’s performance is the extent to which merge decisions interfere with one another. In some inputs, merges are largely independent, while in others, locally optimal choices can destroy multiple future opportunities. Informally, we view merge steps as interacting when performing one merge reduces the availability or utility of others. This perspective provides a unifying explanation for both worst-case and average-case behavior. Taken together, our results suggest that the performance of BPE is governed by the extent to which merge opportunities interfere with one another, pointing toward a more structural understanding of greedy tokenization algorithms.

1.2 Preliminaries

Let Σ be a finite alphabet of size $n = |\Sigma|$, and let $S \in \Sigma^*$ be a string of length $|S|$.

Definition 1.1 (Adjacent pair). *For symbols $x, y \in \Sigma$, an adjacent pair (x, y) is any occurrence of x immediately followed by y in S .*

Remark 1.2. *When counting occurrences of a pair, we consider only non-overlapping occurrences within a single merge step. Following the convention of Kozma and Voderholzer, if a selected pair has overlapping occurrences, we merge the leftmost maximal set of non-overlapping occurrences. In particular, in a substring of the form aaa , only the first occurrence of the pair (a, a) is merged in a single step.*

When several pairs have the same maximum utility, BPE selects the pair whose leftmost occurrence appears earliest in the current string. This convention is used for BPE throughout the paper; the same non-overlapping occurrence convention is used for both BPE and OPT.

1.3 Problem Definition

We study the Byte Pair Encoding (BPE) algorithm as an optimization problem over sequences of merge operations.

Definition 1.3 (Merge step). *A merge step selects a pair (x, y) and replaces all non-overlapping occurrences of (x, y) in the current string with a new symbol $z \notin \Sigma$. The symbol z is then added to the alphabet, and all replacements occur simultaneously.*

The BPE algorithm. Starting from an initial string S , the BPE algorithm iteratively performs merge steps. At each step, it selects a pair (x, y) that maximizes the number of its non-overlapping occurrences in the current string. All non-overlapping occurrences of the selected pair are then replaced simultaneously, producing a new string on which subsequent merge steps operate. If multiple pairs have maximum utility, ties are broken by selecting the pair whose leftmost occurrence appears earliest in the current string.

Utility. The *utility* of a merge step is defined as the number of occurrences of the selected pair at the time the merge is performed. The total utility of a sequence of merge steps is the sum of the utilities of its individual steps.

BPE and OPT. Given a string S and a budget of m merge steps, let $\text{BPE}(S, m)$ denote the total utility achieved by the BPE algorithm after m steps. Let $\text{OPT}(S, m)$

denote the maximum total utility achievable by any sequence of at most m merge steps.

Objective. Our goal is to analyze the approximation ratio $\frac{\text{BPE}(S,m)}{\text{OPT}(S,m)}$, and to understand how structural properties of the input influence this ratio. A key challenge is that merge steps interact. Performing one merge may reduce or eliminate the utility of other potential merges.

Example. Following Kozma and Voderholzer [2], consider $S = \text{aabaaaba}$ with merge budget $m = 3$. Under the leftmost non-overlapping convention, the pairs (a, a) , (a, b) , and (b, a) all have utility 2. Since the leftmost such pair is (a, a) , BPE merges (a, a) first and collects total utility $2 + 1 + 1 = 4$:

$$\text{aabaaaba} \rightarrow \text{XbXaba} \rightarrow \text{YXaba} \rightarrow \text{Zaba}.$$

By contrast, OPT may merge (b, a) first, then (a, a) , and finally the resulting pair (Y, X) , collecting total utility $2 + 2 + 2 = 6$:

$$\text{aabaaaba} \rightarrow \text{aaXaaX} \rightarrow \text{YXYX} \rightarrow \text{ZZ}.$$

Thus, even on a short string, BPE may not achieve the optimal total utility, here gaining only $\frac{2}{3}$ of $\text{OPT}(S, 3)$.

1.4 Related Work

Byte Pair Encoding was introduced to natural language processing by Sennrich, Haddow, and Birch [3] as a simple and effective method for subword tokenization. Since then, it has become a standard preprocessing step in modern NLP systems, including large language models, where it determines how text is segmented into tokens.

Despite its widespread use and practical importance, BPE is typically treated as a heuristic, and its theoretical properties have only recently begun to receive attention.

Kozma and Voderholzer initiated a formal study of BPE as an optimization problem [2], analyzing its performance relative to an optimal sequence of merges. They showed that the worst-case approximation ratio of BPE lies between $\frac{1}{3}$ and $\frac{5}{8}$: BPE always achieves at least a $\frac{1}{3}$ -approximation, while there are instances on which it achieves at most a $\frac{5}{8}$ -approximation. They also showed that computing an optimal merge sequence is APX-hard, further motivating the study of approximation guarantees for BPE. Their work provides the first rigorous guarantees for BPE, but leaves a substantial gap between the known approximation bounds and offers limited insight into the structural properties of inputs that lead to good or bad performance.

A complementary perspective was developed by Zouhar et al. [5], who study BPE under probabilistic assumptions on the input distribution. Their analysis yields lower bounds of the form $\frac{1}{\sigma(\mu^*)} (1 - e^{-\sigma(\mu^*)})$, where $\sigma(\mu^*)$ captures structural properties of the underlying distribution. Empirically, they estimate $\sigma(\mu^*) \approx 2.5$ for synthetic data and $\sigma(\mu^*) \approx 2$ for natural language, in particular English.

While the approach of Zouhar et al. differs significantly from ours, it is notable that their bounds exhibit a similar functional form to the guarantee we obtain in structured settings. Both analyses suggest that BPE’s performance is governed by an underlying parameter controlling the interaction between merge opportunities, providing evidence that the behavior of BPE is driven by structural properties of the input.

1.5 Main Results

We characterize the behavior of BPE across three regimes: worst-case inputs, structured inputs, and random inputs.

Upper bound. We show that BPE cannot achieve better than a $(\frac{1}{2} + \varepsilon)$ -approximation in the worst case (Theorem 2.1).

Lower bound. On structured inputs consisting of length-4 substrings separated by unique delimiters, we show that BPE achieves a $\frac{1-e^{-2}}{2}$ -approximation (Theorem 3.9), providing a nontrivial constant-factor guarantee despite strong interactions between merges.

Random strings. In contrast to the worst-case setting, we develop a conjectural analysis of BPE on uniformly random strings through a random maximum-edge model, supported by experiments. This model predicts that BPE achieves a $(1 - O(1/n))$ -approximation for the first $\frac{n^2}{4}$ merge steps (see Section 4).

Chapter 2

Upper Bound on the BPE Approximation Factor

In this section, we construct a family of instances on which BPE achieves at most a $(\frac{1}{2} + \varepsilon)$ -approximation for any $\varepsilon > 0$.

Theorem 2.1. *For every $\varepsilon > 0$, there exists an input on which BPE is at most a $(\frac{1}{2} + \varepsilon)$ -approximation.*

Proof.

Construction

We consider strings composed of length-4 substrings separated by delimiter symbols. Each delimiter is a unique symbol that appears exactly once in the string, and we denote delimiters by $|$. Since any adjacent pair containing a delimiter occurs at most once, no merge selected by BPE can cross a delimiter boundary. Thus all merges are restricted to occur within individual length-4 substrings.

Let the alphabet consist of symbols $\{a_i, b_j, c_k, d_\ell \mid i, j, k, \ell \in \{1, 2, \dots, M\}\}$, where all symbols are distinct. We form a base string containing one copy of every length-4 substring $a_i b_j c_k d_\ell$ for all choices of i, j, k, ℓ , separated by unique delimiters. We then

concatenate K copies of this base string.

$$a_1b_1c_1d_1 \mid a_2b_1c_1d_1 \mid a_2b_2c_1d_1 \mid \cdots \mid a_Mb_Mc_Md_M \mid$$

Thus, for every pair (i, j) , the pair a_ib_j appears exactly KM^2 times, since it is paired with all M^2 choices of (k, ℓ) . Similarly, each pair b_jc_k and each pair c_kd_ℓ also appears exactly KM^2 times.

To bias BPE toward the middle pair, we append one additional “spoiler” substring b_jc_k for every choice of $j, k \in \{1, \dots, M\}$, again separated by unique delimiters. As a result, each middle pair b_jc_k now appears $KM^2 + 1$ times, while each left pair a_ib_j and each right pair c_kd_ℓ still appears exactly KM^2 times. Finally, we set the merge budget to $m = 2M^2$.

$$a_1b_1c_1d_1 \mid a_2b_1c_1d_1 \mid a_2b_2c_1d_1 \mid \cdots \mid a_Mb_Mc_Md_M \mid b_1c_1 \mid b_2c_1 \mid \cdots \mid b_Mc_M$$

Behavior of BPE

Because each pair b_jc_k occurs $KM^2 + 1$ times, while every pair a_ib_j and c_kd_ℓ occurs only KM^2 times, BPE must spend its first M^2 steps merging all middle pairs b_jc_k .

Each such merge contributes utility $KM^2 + 1$, so after the first phase BPE has collected total utility $M^2(KM^2 + 1) = KM^4 + M^2$. After these merges, each length-4 substring has the form $a_iX_{jk}d_\ell$, where X_{jk} denotes the merged token created from b_jc_k . In the remaining M^2 steps, BPE can only merge pairs of the form a_iX_{jk} or $X_{jk}d_\ell$. Each such pair has utility at most KM . Therefore, regardless of the order in which these tied pairs are selected, the second phase contributes at most $M^2 \cdot KM = KM^3$.

For a fixed pair (i, jk) , the pair a_iX_{jk} appears once for each choice of ℓ , and hence appears exactly KM times across the string. Therefore, each of the remaining M^2 merges contributes utility KM , and the second phase contributes total utility $M^2 \cdot KM = KM^3$. Hence the total utility of BPE is $U_{\text{BPE}} = KM^4 + KM^3 + M^2$.

Behavior of the Optimal Solution

Consider instead the strategy that first merges all left pairs $a_i b_j$. Since each such pair appears KM^2 times, the first M^2 steps contribute total utility $M^2 \cdot KM^2 = KM^4$. After this, each substring has the form $Y_{ij} c_k d_\ell$, where Y_{ij} denotes the token formed from $a_i b_j$. The right pairs $c_k d_\ell$ are still intact, and each such pair also appears KM^2 times. Thus OPT may spend the next M^2 steps merging all right pairs $c_k d_\ell$, gaining an additional KM^4 utility. Therefore the optimal solution achieves total utility $U_{\text{OPT}} = 2KM^4$.

Approximation Ratio. Combining the two computations, we obtain

$$\frac{U_{\text{BPE}}}{U_{\text{OPT}}} = \frac{KM^4 + KM^3 + M^2}{2KM^4} = \frac{1}{2} + \frac{1}{2M} + \frac{1}{2KM^2}.$$

For fixed M , and for large K this gives

$$\frac{U_{\text{BPE}}}{U_{\text{OPT}}} = \frac{1}{2} + \frac{1}{2M}.$$

Since M may be chosen arbitrarily large, this implies that for every $\varepsilon > 0$, there exists an instance on which BPE achieves at most a $(\frac{1}{2} + \varepsilon)$ -approximation. $\square$

Chapter 3

Lower Bound on BPE Approximation Factor

In this chapter, we prove a lower bound for BPE on inputs consisting of length-4 blocks separated by unique delimiter symbols. Let the input string be S , and let the merge budget be m . Recall that $\text{BPE}(S, m)$ denotes the total utility collected by BPE after m merge steps, while $\text{OPT}(S, m)$ denotes the maximum total utility achievable by any sequence of at most m merge steps.

Since the input consists of length-4 blocks separated by unique delimiters, no merge of utility greater than one can cross a delimiter boundary. Thus all relevant merge interactions occur inside individual length-4 blocks.

We decompose an optimal merge sequence into two phases. The first phase consists of all merge steps that create tokens of lengths 2 and 3. The second phase consists of merge steps that create fully merged length-4 tokens. By Lemma 3.5, any merge that creates a fully merged length-4 token can be postponed without affecting the feasibility or utility of later merges. Therefore, we may reorder an optimal sequence so that all full length-4 merges occur at the end.

Let m_1 be the number of first-phase merges in this reordered optimal sequence,

and let m_2 be the number of second-phase merges. Thus $m = m_1 + m_2$. We write OPT_{m_1} for the multiset of merge occurrences collected by OPT during its first m_1 steps, and OPT_{m_2} for the multiset of merge occurrences collected by OPT during its latter m_2 steps.

We do not reorder the BPE sequence. We simply compare the first m_1 steps of BPE to the first phase of OPT, and then compare the remaining BPE steps to the second phase of OPT.

The first comparison uses a residual accounting argument. Each concrete BPE occurrence can destroy at most two first-phase OPT occurrences, which yields a recurrence lower-bounding the utility collected by BPE. The second comparison uses the fact that full length-4 completions can be postponed, allowing common completed tokens to be aligned and removed before charging the remaining OPT completions to distinct BPE steps.

3.1 First m_1 steps

We first analyze the first m_1 steps, where OPT creates only tokens of lengths 2 and 3. Since fully merging a length-4 block requires three internal merge occurrences, every block contributes at most two merge occurrences to OPT_{m_1} .

Throughout this section, we reason occurrence-by-occurrence, using the non-overlapping occurrence convention from Remark 1.2.

Lemma 3.1. *Let OPT_{m_1} denote the multiset of merge occurrences collected by OPT during its first m_1 steps, after reordering so that these steps create only tokens of lengths 2 and 3. Let BPE_i denote the total utility collected by BPE after i steps.*

For each original length-4 block, consider the local first-phase pattern used by OPT inside that block. Call a block compatible after i BPE steps if the BPE occurrences performed inside that block are all occurrences from the local first-phase OPT pattern,

and no dependent occurrence has been performed before its prerequisite. Otherwise, call the block spoiled. Let R_i be the set of occurrences in OPT_{m_1} that lie in compatible blocks and have not already been performed by BPE. Then

$$|R_i| \geq |\text{OPT}_{m_1}| - 2\text{BPE}_i.$$

Proof. Since the first m_1 steps of OPT create only tokens of lengths 2 and 3, no original length-4 block is fully merged during this phase. Fully merging a length-4 block requires three internal merge occurrences. Therefore each block contributes at most two occurrences to OPT_{m_1} .

Each unit of BPE utility corresponds to one concrete merge occurrence inside one original length-4 block. If this occurrence agrees with the local OPT pattern in that block, it removes from R_i only the corresponding OPT occurrence that has now been performed. If it does not agree with the local OPT pattern, then the block becomes spoiled, and all remaining first-phase OPT occurrences in that block are discarded from the accounting.

In either case, the affected block contains at most two first-phase OPT occurrences. Hence each concrete BPE occurrence removes at most two occurrences from the residual accounting. Since BPE has performed exactly BPE_i concrete merge occurrences after i steps, at most 2BPE_i occurrences have been removed. Therefore

$$|R_i| \geq |\text{OPT}_{m_1}| - 2\text{BPE}_i.$$

□

Lemma 3.2. *After $i < m_1$ steps of BPE, there exists an available merge in the current BPE string whose utility is at least $\frac{|R_i|}{m_1}$.*

Proof. Fix the BPE string after i steps. Recall that R_i is the set of residual first-phase

OPT occurrences lying in compatible blocks. We call an occurrence in R_i *active* if all of its local prerequisites, according to the OPT pattern inside its block, have already been performed by BPE. Since the block is compatible, every active occurrence is currently available in the BPE string.

Let F_i denote the set of merge types that have at least one active occurrence after i BPE steps. Thus each $f \in F_i$ is an available pair type in the current BPE string. For $f \in F_i$, let $u_i(f)$ be the number of active occurrences of type f . Since these occurrences are present in the current BPE string, the true utility of the merge type f is at least $u_i(f)$.

We now charge each occurrence in R_i to an active occurrence in the same block. If the occurrence is active, we charge it to itself. If the occurrence is not active, then it must be the second merge in a dependent two-merge first-phase pattern inside a length-4 block. In this case, we charge it to its unique prerequisite occurrence, which is active.

For each active merge type $f \in F_i$, let $D(f)$ be the set of first-phase OPT merge steps that have at least one residual occurrence in a compatible block charged to an active occurrence of type f . We claim that each first-phase OPT merge step belongs to at most one set $D(f)$.

To see this, fix a first-phase OPT merge step s . If s is primary, meaning that its occurrences have no local prerequisite inside their blocks, then every residual occurrence of s is active and is charged only to occurrences of type s . Hence s belongs to at most one set $D(f)$.

If s is secondary, then within each compatible block it has a unique primary prerequisite p . Before BPE performs the merge type corresponding to p , every residual occurrence of s is charged to an active occurrence of type p . Once BPE performs this merge type, the global nature of BPE together with the fixed leftmost non-overlapping convention implies that all compatible occurrences of p have been performed. At that

point, every remaining residual occurrence of s becomes active and is charged only to type s . Therefore, at any fixed time after i BPE steps, the residual occurrences of s are charged to only one active type. Thus s belongs to at most one set $D(f)$.

It follows that

$$\sum_{f \in F_i} |D(f)| \leq m_1,$$

since there are only m_1 first-phase OPT merge steps.

Now consider the amount of residual mass charged to a fixed active type $f \in F_i$. For each OPT step $s \in D(f)$, at most one residual occurrence of s can be charged to each active occurrence of type f . Hence the number of residual occurrences of s charged to type f is at most $u_i(f)$. Therefore the total number of residual occurrences charged to type f is at most

$$|D(f)|u_i(f).$$

Since every occurrence in R_i is charged to some active type, we have

$$|R_i| \leq \sum_{f \in F_i} |D(f)|u_i(f).$$

If every active type $f \in F_i$ satisfied

$$u_i(f) < \frac{|R_i|}{m_1},$$

then

$$|R_i| \leq \sum_{f \in F_i} |D(f)|u_i(f) < \frac{|R_i|}{m_1} \sum_{f \in F_i} |D(f)| \leq |R_i|,$$

a contradiction. Therefore, there exists an active type $f \in F_i$ such that

$$u_i(f) \geq \frac{|R_i|}{m_1}.$$

Since the true utility of type f in the current BPE string is at least $u_i(f)$, there exists

an available merge of utility at least $|R_i|/m_1$. $\square$

Lemma 3.3. *After $i < m_1$ steps of BPE, there exists an available merge in the current BPE string whose utility is at least*

$$\frac{|\text{OPT}_{m_1}| - 2\text{BPE}_i}{m_1}.$$

Proof. By Lemma 3.1,

$$|R_i| \geq |\text{OPT}_{m_1}| - 2\text{BPE}_i.$$

By Lemma 3.2, there exists an available merge of utility at least

$$\frac{|R_i|}{m_1}.$$

Therefore there exists an available merge of utility at least

$$\frac{|\text{OPT}_{m_1}| - 2\text{BPE}_i}{m_1}.$$

$\square$

Lemma 3.4. *The utility collected by BPE during its first m_1 steps satisfies*

$$\text{BPE}_{m_1} \geq \frac{1 - e^{-2}}{2} |\text{OPT}_{m_1}|.$$

Proof. By Lemma 3.3, after $i < m_1$ steps of BPE, there is an available merge of utility at least

$$\frac{|\text{OPT}_{m_1}| - 2\text{BPE}_i}{m_1}.$$

Since BPE chooses a maximum-utility available merge, its next merge has utility at

least this amount. Therefore

$$\text{BPE}_{i+1} \geq \text{BPE}_i + \frac{|\text{OPT}_{m_1}| - 2\text{BPE}_i}{m_1}.$$

Equivalently,

$$\text{BPE}_{i+1} \geq \left(1 - \frac{2}{m_1}\right) \text{BPE}_i + \frac{|\text{OPT}_{m_1}|}{m_1}.$$

Since $\text{BPE}_0 = 0$, repeated application of this recurrence gives

$$\text{BPE}_{m_1} \geq \frac{|\text{OPT}_{m_1}|}{m_1} \sum_{j=0}^{m_1-1} \left(1 - \frac{2}{m_1}\right)^j.$$

Evaluating the geometric sum,

$$\text{BPE}_{m_1} \geq \frac{|\text{OPT}_{m_1}|}{2} \left(1 - \left(1 - \frac{2}{m_1}\right)^{m_1}\right).$$

Using

$$\left(1 - \frac{2}{m_1}\right)^{m_1} \leq e^{-2},$$

we obtain

$$\text{BPE}_{m_1} \geq \frac{1 - e^{-2}}{2} |\text{OPT}_{m_1}|.$$

□

3.2 Latter m_2 steps

We now analyze the latter m_2 steps of OPT , namely those steps that create fully merged length-4 tokens.

Let M_{OPT} and M_{BPE} denote the merge sequences of OPT and BPE , respectively. Recall that, after reordering OPT , the first m_1 steps of M_{OPT} create only tokens of lengths 2 and 3, while the final m_2 steps create fully merged length-4 tokens. Let

$M_{\text{OPT}}^{(2)}$ denote this latter collection of m_2 merge steps.

Lemma 3.5. *Assume the input consists of length-4 substrings separated by unique delimiter symbols, so that merges never cross substring boundaries. Let t be a merge step that creates a fully merged length-4 token. Then t can be moved to any later position in the merge sequence without changing the utility or feasibility of any other later merge step.*

Proof. Let t be a merge step creating a fully merged length-4 token T . Since T occupies an entire length-4 substring, and since substring boundaries are separated by unique delimiters, the token T cannot participate in any later merge across a boundary. Moreover, because T already represents the entire substring, there is no remaining merge internal to that substring.

Therefore, once T is created, no later merge step can involve T . It follows that the only effect of t on the remainder of the merge sequence is to contribute its own utility and consume one step of budget. In particular, t does not alter the availability or utility of any later merge step.

Hence t commutes with every later merge step. Swapping t with the step immediately following it preserves the resulting sequence of later utilities. Repeating this argument, t may be moved to any later position in the sequence. $\square$

The lemma implies that any merge in $M_{\text{OPT}}^{(2)}$ may be shifted later without affecting the rest of the sequence. We use this to align all fully merged length-4 tokens that appear in both OPT and BPE.

Let

$$I = M_{\text{OPT}}^{(2)} \cap M_{\text{BPE}}$$

denote the set of fully merged length-4 tokens that occur in both solutions. For each $t \in I$, Lemma 3.5 allows us to move the occurrence of t in M_{OPT} to a position later than or equal to the position where t appears in M_{BPE} . Since t is the same completed

token in both sequences and contributes the same utility in both, we may delete t from both sequences and reduce the merge budget by one.

Repeating this for every $t \in I$, we obtain reduced merge sequences M'_{OPT} and M'_{BPE} with the following properties:

1. Every remaining second-phase merge in M'_{OPT} creates a fully merged length-4 token.
2. No remaining second-phase fully merged length-4 token of M'_{OPT} appears in M'_{BPE} .
3. Removing the common completed tokens subtracts the same total utility from both solutions.

It therefore suffices to compare the reduced sequences.

Lemma 3.6. *Let T be a token, and let t be any substring token used in a merge sequence that produces T . Then the number of occurrences of t is at least the number of occurrences of T . Consequently, any merge step used to create t has utility at least that of any later merge step used to create T .*

Proof. Every occurrence of T contains an occurrence of t . Hence the number of occurrences of t is at least the number of occurrences of T . Since merge utility is defined as the number of occurrences of the merged pair at the moment the merge is performed, any merge creating a precursor substring has utility at least that of a later merge creating the full token. $\square$

Observation 3.7. *Fix an original length-4 block type W . Under the standing assumption that merges do not cross delimiter boundaries, all occurrences of W evolve identically under BPE until they are fully merged.*

Proof. Initially, all occurrences of W have the same internal tokenization. Since delimiters are unique and separate the length-4 blocks, no merge under consideration

crosses a delimiter boundary. Thus the effect of BPE on an occurrence of W depends only on the internal tokenization of that occurrence.

At each step, BPE selects a merge type globally and applies it simultaneously to all non-overlapping occurrences of that type, using the fixed leftmost non-overlapping convention. Therefore, if two occurrences of W have the same internal tokenization before a BPE step, they also have the same internal tokenization after that step. By induction over BPE steps, all occurrences of W evolve identically until they are fully merged. $\square$

Lemma 3.8. *Let g_W be a second-phase OPT merge that completes occurrences of an original length-4 block type W , and let $q = |g_W|$ be its utility. If BPE has not already fully merged the occurrences of W completed by g_W , then in the current BPE string there exists an available merge of utility at least q .*

Proof. The merge g_W completes q occurrences of the original block type W . By Observation 3.7, these occurrences evolve identically under BPE until they are fully merged. Since BPE has not already fully merged them, each such occurrence is currently represented internally by either four tokens, three tokens, or two tokens.

If each occurrence of W is represented by two tokens, then those two tokens are adjacent and together form the entire block W . Therefore the merge completing W is available in all q occurrences. Hence there is an available merge of utility at least q .

Otherwise, each occurrence of W is represented by either four tokens or three tokens. In this case, at least one internal submerge of W remains available. Let p be such an available submerge. Because all q occurrences of W have the same internal tokenization, the same submerge p is available in all q occurrences. By Lemma 3.6, the utility of this available submerge is at least the utility of the eventual full-block completion. Thus $|p| \geq q$. Therefore, whether BPE can already complete W or can only perform an internal submerge of W , there exists an available merge of utility

at least $q = |g_W|$. Since BPE chooses a maximum-utility available merge, the merge chosen by BPE has utility at least $|g_W|$. $\square$

We now compare the remaining second-phase merges of OPT to distinct BPE steps.

After aligning and removing the fully merged length-4 block types that occur in both OPT and BPE, consider the remaining second-phase OPT merges. Each such merge completes some original length-4 block type W that BPE has not already fully merged in the reduced comparison.

Let these remaining OPT completion merges be

$$g_{W_1}, g_{W_2}, \dots, g_{W_r},$$

listed in the order in which they occur in OPT's second phase. Let $q_j = |g_{W_j}|$ be the utility of the j -th such merge.

We compare these OPT completion merges to the corresponding remaining BPE steps. At the moment of the j -th comparison, BPE has not already completed the block type W_j in the reduced comparison; otherwise that completed block type would have been aligned and removed from both sequences. Therefore, by Lemma 3.8, BPE has an available merge of utility at least q_j . Since BPE is greedy, the merge it actually performs at this step has utility at least q_j . Thus the j -th remaining OPT completion can be charged to a distinct BPE step of at least equal utility.

Summing over all remaining second-phase OPT completions gives

$$\sum_{j=1}^r |g_{W_j}| \leq \sum_{j=1}^r |\text{corresponding BPE step}_j|.$$

Therefore, after common completed length-4 block types have been removed symmetrically, BPE loses no additional utility relative to OPT during the second phase.

Theorem 3.9. *Assume the input consists of length-4 substrings separated by unique delimiter symbols. Then*

$$\text{BPE}(S, m) \geq \frac{1 - e^{-2}}{2} \text{OPT}(S, m).$$

Equivalently, BPE is a $\frac{1-e^{-2}}{2}$ -approximation on this class of inputs.

Proof. Let M_{OPT} and M_{BPE} denote the merge sequences of OPT and BPE, respectively. By the phase decomposition above, we may write the reordered execution of OPT as consisting of two parts: the first m_1 steps, which create only tokens of lengths 2 and 3, and the latter m_2 steps, which create fully merged length-4 tokens.

By Lemma 3.4, the utility collected by BPE during its first m_1 steps satisfies

$$\text{BPE}_{m_1} \geq \frac{1 - e^{-2}}{2} |\text{OPT}_{m_1}|.$$

By the second-phase comparison above, after common completed length-4 tokens are aligned and removed, BPE loses no additional utility relative to OPT during the latter m_2 steps. Therefore the utility collected by BPE during the remaining steps is at least $|\text{OPT}_{m_2}|$ in the reduced comparison.

Hence

$$\text{BPE}(S, m) \geq \frac{1 - e^{-2}}{2} |\text{OPT}_{m_1}| + |\text{OPT}_{m_2}|.$$

Since

$$\frac{1 - e^{-2}}{2} \leq 1,$$

we have

$$\frac{1 - e^{-2}}{2} |\text{OPT}_{m_1}| + |\text{OPT}_{m_2}| \geq \frac{1 - e^{-2}}{2} (|\text{OPT}_{m_1}| + |\text{OPT}_{m_2}|).$$

But

$$|\text{OPT}_{m_1}| + |\text{OPT}_{m_2}| = \text{OPT}(S, m).$$

Therefore

$$\text{BPE}(S, m) \geq \frac{1 - e^{-2}}{2} \text{OPT}(S, m).$$

□

Chapter 4

Tokenizing a Uniformly Random String

In this section, we analyze the behavior of BPE on uniformly random strings. Let Σ be an alphabet of size n , and let S be a string of length $|S|$ where each character is drawn independently and uniformly at random from Σ .

Under this model, the expected number of occurrences of any symbol is $\frac{|S|}{n}$, and the expected number of occurrences of any adjacent pair (x, y) is $\frac{|S|}{n^2}$. For sufficiently large $|S|$, these quantities are tightly concentrated, and we treat all pairs as having approximately equal frequency.

4.1 Optimal Strategy

We first describe a strategy that achieves the maximum possible utility for a large number of steps.

Lemma 4.1. *There exists a sequence of $\frac{n^2}{4}$ merge steps such that each step achieves utility $\frac{|S|}{n^2}$.*

Proof. Assume n is even. Partition Σ into two disjoint sets Σ_1 and Σ_2 of equal size.

Consider all pairs (a, b) where $a \in \Sigma_1$ and $b \in \Sigma_2$.

Merging all such pairs yields $\frac{n^2}{4}$ merge steps. Since no symbol in Σ_1 appears as a right character and no symbol in Σ_2 appears as a left character in any merge, these merges do not interfere with each other. Thus, each pair (a, b) continues to occur with frequency $\frac{|S|}{n^2}$ throughout the sequence, and each merge achieves maximum utility. $\square$

4.2 Graph Structure of BPE

We now analyze the behavior of BPE. We represent the merge process as a directed graph $G = (V, E)$, where $V = \Sigma$, and each merge (x, y) corresponds to adding a directed edge from x to y .

As long as BPE selects pairs of maximum frequency, the following structural property holds.

Lemma 4.2. *During the phase in which BPE selects only maximum-frequency pairs, each original symbol can appear only as a left endpoint of selected merges or only as a right endpoint of selected merges, but not both.*

Proof. Let the input string S be drawn uniformly at random from an alphabet Σ of size n , and assume $|S|$ is sufficiently large so that all relevant pair and triple counts are tightly concentrated around their expectations.

Initially, every adjacent pair has expected frequency approximately $\frac{|S|}{n^2}$. Suppose BPE performs a merge (x, y) , producing a new symbol A . We claim that, after this merge, pairs using x as a right endpoint or y as a left endpoint are no longer maximum-frequency pairs.

Consider a pair of the form (z, x) . Before the merge, its expected frequency is approximately $\frac{|S|}{n^2}$. The occurrences of (z, x) destroyed by the merge (x, y) are precisely those appearing inside triples (z, x, y) , since these become occurrences of (z, A) after xy is replaced by A . The expected number of such triples is approximately

$\frac{|S|}{n^3}$. Thus, after the merge, the expected frequency of (z, x) is approximately $\frac{|S|}{n^2} - \frac{|S|}{n^3} = \frac{|S|}{n^2} \left(1 - \frac{1}{n}\right)$. The same argument applies symmetrically to pairs of the form (y, z) .

In contrast, any pair whose symbols have not been affected by prior merges remains at expected frequency approximately $\frac{|S|}{n^2}$. Therefore, pairs of the form (z, x) and (y, z) have expected utility lower than untouched pairs by about $\frac{|S|}{n^3}$.

For sufficiently large $|S|$, standard Chernoff bounds imply that all relevant pair and triple counts are concentrated tightly enough around their expectations that this gap persists with high probability. Hence, while untouched maximum-frequency pairs remain available, BPE will not select a pair of the form (z, x) after selecting (x, y) , nor a pair of the form (y, z) .

It follows that once an original symbol appears as the left endpoint of a selected maximum-frequency merge, it cannot later appear as the right endpoint of another maximum-frequency merge. Similarly, once it appears as the right endpoint of a selected maximum-frequency merge, it cannot later appear as the left endpoint. Thus, during the maximum-frequency phase, each original symbol appears only on one side of selected merges. $\square$

Remark 4.3. *Note that the expected number of occurrences of the pair (a, a) is $\frac{|S|(n-1)}{n^3}$, and thus is strictly smaller than the expected frequency $\frac{|S|}{n^2}$ of a typical pair (x, y) with $x \neq y$. Consequently, as long as a merge of utility $\frac{|S|}{n^2}$ is available, BPE will not select the pair (a, a) .*

This discrepancy arises because BPE considers only non-overlapping occurrences within a merge step. In particular, in a substring of the form “aaa”, the two overlapping occurrences of (a, a) cannot both be counted simultaneously, and only one can be merged in a single step. This reduces the effective frequency of (a, a) relative to non-overlapping pairs. See [5] for a formal treatment of this behavior.

As a consequence, the vertex set Σ can be partitioned into two sets:

$$V_1 = \{x : x \text{ appears as a left character}\}, \quad V_2 = \{x : x \text{ appears as a right character}\}.$$

All edges are directed from V_1 to V_2 , and thus BPE induces a directed bipartite graph.

It follows that BPE continues selecting maximum-utility merges until all edges between V_1 and V_2 have been added. The number of such merges is $|V_1||V_2|$.

4.3 Imbalance of the Bipartition

Let $|V_1| = a$ and $|V_2| = b$, and define the imbalance

$$D = a - b.$$

Then the number of maximum-utility merges selected by BPE is

$$|V_1||V_2| = \left(\frac{n+D}{2}\right)\left(\frac{n-D}{2}\right) = \frac{n^2 - D^2}{4}.$$

To estimate this quantity, it remains to understand the behavior of D as vertices are assigned to V_1 and V_2 .

4.4 Conjectural Models

We do not currently prove the exact distribution governing how symbols are assigned to V_1 and V_2 . However, the symmetry of the random input suggests the following natural conjectures.

Conjecture 4.4 (Uniform assignment). *Conditioned on the current stage of the process, a previously unassigned symbol is equally likely to be added to either V_1 or V_2 .*

This conjecture reflects a weaker symmetry assumption. Since the input string is fully uniform and all pairs initially have the same expected frequency, there is no inherent bias favoring either side of the partition. Ignoring the dependence on the current sizes of V_1 and V_2 , it is therefore natural to model the assignment of symbols as an unbiased random process.

Conjecture 4.5 (Balanced assignment). *Conditioned on the current partition (V_1, V_2) with $|V_1| = a$ and $|V_2| = b$, the next previously unassigned symbol is added to V_1 with probability $\frac{b}{a+b}$ and to V_2 with probability $\frac{a}{a+b}$.*

This conjecture is motivated by the observation that, in a uniformly random string, all pairs have approximately equal frequency, so BPE effectively selects among many candidate pairs with no strong bias. Among such pairs, a symbol that has not yet been assigned is more likely to appear in a pair with a symbol from the larger side of the partition simply because there are more such symbols available. Thus, the probability of joining a side is proportional to the size of the opposite side.

Conjecture 4.6 (Random maximum-edge). *Conditioned on the current partition (V_1, V_2, U) , where U is the set of unassigned symbols, BPE selects uniformly among all currently available maximum-utility pairs that involve at least one symbol in U .*

This conjecture refines the previous models by accounting for the fact that BPE selects pairs rather than individual symbols. In particular, a merge may involve two previously unassigned symbols, in which case one symbol is added to V_1 and the other to V_2 , leaving the imbalance unchanged. This effect is especially pronounced early in the process, where most maximum-utility pairs consist entirely of unassigned symbols.

4.5 Consequences of the Conjectures

Let D_t denote the imbalance after t symbols have been assigned.

Uniform assignment. Under Conjecture 1, each new symbol is equally likely to be assigned to either side of the partition. Thus

$$D_{i+1} = D_i \pm 1 \quad \text{with probability } \frac{1}{2},$$

and so

$$\mathbb{E}[D_{i+1}^2 \mid D_i] = \frac{(D_i + 1)^2 + (D_i - 1)^2}{2} = D_i^2 + 1.$$

Simplifying and taking expectations gives

$$\mathbb{E}[D_{i+1}^2] = \mathbb{E}[D_i^2] + 1.$$

With the same base case $\mathbb{E}[D_3^2] = 1$, it follows by induction that

$$\mathbb{E}[D_t^2] = t - 2.$$

Balanced assignment. Under Conjecture 2, if $|V_1| = a$ and $|V_2| = b$ after i assignments, then $a + b = i$. Moreover, the next unassigned symbol joins V_1 with probability $\frac{b}{a+b}$ and joins V_2 with probability $\frac{a}{a+b}$, so

$$\mathbb{E}[D_{i+1}^2 \mid D_i] = (D_i + 1)^2 \frac{b}{a+b} + (D_i - 1)^2 \frac{a}{a+b}.$$

Substituting $a + b = i$ and $D_i = a - b$ gives

$$\mathbb{E}[D_{i+1}^2 \mid D_i] = D_i^2 \left(1 - \frac{2}{i}\right) + 1.$$

Evaluating the recurrence yields

$$\mathbb{E}[D_t^2] = \frac{t}{3}.$$

Random maximum-edge. Under Conjecture 3, let $a = |V_1|$, $b = |V_2|$, $r = n - a - b$, and $D = a - b$. The possible transitions are

$$D_{j+1} = \begin{cases} D_j, & \text{with probability } \frac{r-1}{n-1}, \\ D_j + 1, & \text{with probability } \frac{b}{n-1}, \\ D_j - 1, & \text{with probability } \frac{a}{n-1}. \end{cases}$$

Therefore,

$$\mathbb{E}[D_{j+1}^2 \mid D_j] = D_j^2 \frac{r-1}{n-1} + (D_j + 1)^2 \frac{b}{n-1} + (D_j - 1)^2 \frac{a}{n-1}.$$

Using $a + b = t$, $r = n - a - b$, and $D = a - b$, this simplifies to

$$\mathbb{E}[D_{j+1}^2 \mid D_j, i] = D_j^2 \left(1 - \frac{2}{n-1}\right) + \frac{i}{n-1}.$$

From here, we approximate this as a continuous process for large enough n . Note that unlike conjectures 1 and 2, this conjecture can have anywhere from $n/2$ to n steps. So we create $M(i) = \mathbb{E}[D_i^2]$ to represent how our expectation of D changes after i steps, and then divide by the expected amount of new symbols per step. We derive in Appendix A that we can represent this change as

$$\frac{\Delta M}{\Delta i} = \frac{i - 2M}{2n - i} \approx \frac{dM}{di}.$$

We then subsequently solve this differential equation from 0 to n to find the expectation of the function and our result is

$$\mathbb{E}[D_n^2] = \frac{n}{4}.$$

4.6 Approximation Guarantees

The preceding calculations estimate the expected imbalance $D = |V_1| - |V_2|$ under each conjectural model. Once D is known, the number of maximum-utility merges selected by BPE is

$$|V_1||V_2| = \binom{n+D}{2} \binom{n-D}{2} = \frac{n^2 - D^2}{4}.$$

Therefore,

$$\mathbb{E}[|V_1||V_2|] = \frac{n^2}{4} - \frac{\mathbb{E}[D^2]}{4}.$$

The three conjectural models give the following estimates:

Model	$\mathbb{E}[D^2]$	$\mathbb{E}[V_1 V_2]$
Uniform assignment	n	$\frac{n^2}{4} - \frac{n}{4}$
Balanced assignment	$\frac{n}{3}$	$\frac{n^2}{4} - \frac{n}{12}$
Random maximum-edge	$\frac{n}{4}$	$\frac{n^2}{4} - \frac{n}{16}$

Since each maximum-utility merge contributes approximately $\frac{|S|}{n^2}$ utility, while the optimal strategy achieves $\frac{n^2}{4}$ such merges, each model implies a near-optimal approximation ratio of the form

$$1 - O\left(\frac{1}{n}\right).$$

In particular, the random maximum-edge model predicts

$$\frac{\mathbb{E}[\text{BPE}]}{\text{OPT}} \approx 1 - \frac{1}{4n}.$$

Thus, under these conjectural models, BPE achieves near-optimal performance on uniformly random strings for $\Theta(n^2)$ merge steps.

Chapter 5

Experimental Results

We empirically evaluate the behavior of BPE on uniformly random strings to support the predictions of the random maximum-edge model developed in Section 4. In particular, we test two predictions. First, the number of maximum-utility, or “perfect,” merges before the first conflict should be close to $\frac{n^2}{4} - \frac{n}{16}$. Second, the partition induced by BPE during the maximum-utility phase should remain strongly concentrated around a balanced split.

5.1 Setup

For each alphabet size n , we generate 100 random strings of length 100,000 by drawing characters independently and uniformly from this alphabet. We then run BPE and record two quantities.

The first quantity is the *conflict spot*: the first merge step at which BPE is forced to select a pair with utility strictly smaller than the initial maximum frequency. Equivalently, the conflict spot is one plus the number of perfect merges.

The second quantity is the *partition split*. During the maximum-utility phase, the selected merges induce a bipartition of the alphabet into symbols appearing on the left side of selected merges and symbols appearing on the right side. We record the

signed imbalance $|V_1| - |V_2|$. A perfectly balanced split has imbalance 0, while larger positive or negative values indicate a more uneven partition.

To make the experiments reproducible, the code used to generate the random strings, run BPE, compute the conflict spots and partition imbalances, and produce the figures in this chapter is available in the accompanying GitHub repository [4].

5.2 Results

Figure 5.1 compares the observed average number of merges before the first conflict to the value predicted by the random maximum-edge model. Across all tested alphabet sizes, the observed values are extremely close to the prediction

$$\frac{n^2}{4} - \frac{n}{16}.$$

This supports the claim that BPE behaves like the random maximum-edge process during the initial maximum-utility phase.

Figure 5.2 shows the distribution of signed partition imbalances. The distributions are sharply concentrated near zero, indicating that the bipartitions produced by BPE are typically close to balanced. This is consistent with the prediction that

$$\mathbb{E}[D^2] \approx \frac{n}{4},$$

where $D = |V_1| - |V_2|$. In particular, the largest mass occurs at or near the most balanced splits, and large imbalances occur rarely.

Together, these experiments provide empirical support for the random maximum-edge model. The first figure shows that the predicted number of perfect merges matches the observed number closely, while the second figure explains why. BPE tends to produce a balanced bipartition, and therefore achieves nearly the maximum

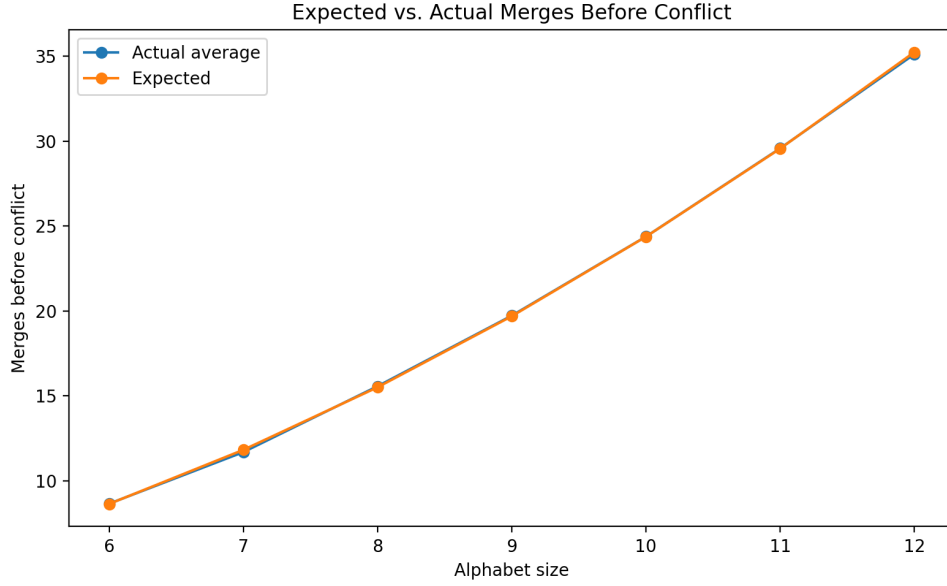


Figure 5.1: Observed average number of merges before the first conflict compared with the prediction of the random maximum-edge model. The empirical averages closely track the predicted values across the tested alphabet sizes.

possible number of noninterfering merges.

5.3 Discussion

The experimental results suggest that the random maximum-edge model captures the early behavior of BPE on uniformly random strings with surprising accuracy. The number of perfect merges is almost indistinguishable from the predicted value, and the observed partition imbalances are tightly concentrated around balanced splits.

This provides evidence that, on uniformly random inputs, BPE avoids the destructive merge interactions that drive the worst-case examples. In the worst-case construction, locally attractive merges can eliminate multiple future high-value opportunities. By contrast, in the random setting, the merge opportunities are distributed more symmetrically. BPE’s greedy choices therefore tend to organize the alphabet into an approximately balanced bipartition, allowing it to collect nearly as much utility as the optimal noninterfering strategy for the first $\Theta(n^2)$ steps.

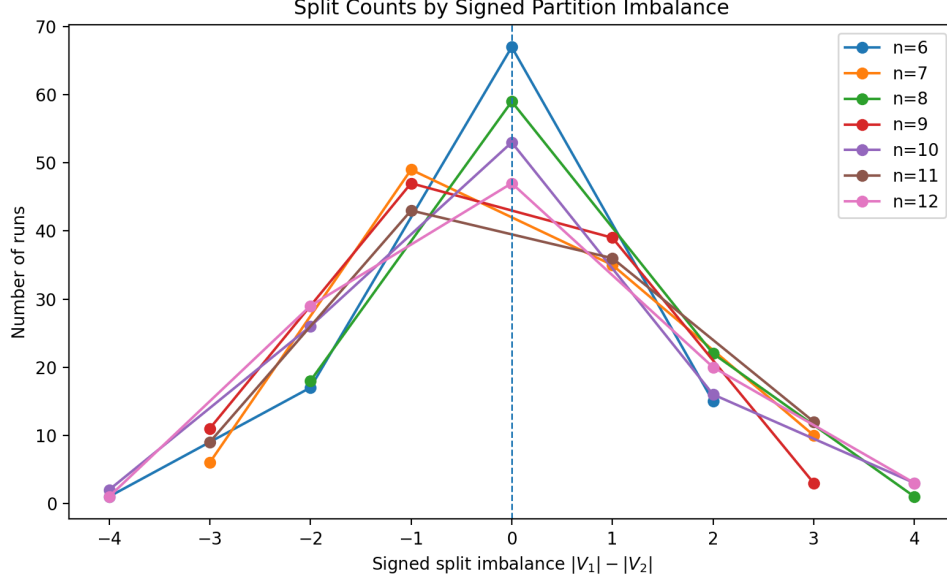


Figure 5.2: Distribution of signed partition imbalances $|V_1| - |V_2|$ across random trials. The distributions are concentrated near zero, showing that BPE usually induces a nearly balanced bipartition during the maximum-utility phase.

The concentration of the split distributions also suggests a possible explanation for why BPE may continue to perform well even beyond the first $n^2/4$ merge steps. Although the theoretical analysis in Section 4 focuses on the initial maximum-utility phase, the empirical behavior indicates that BPE has not merely collected many high-utility merges by chance. Rather, it appears to preserve a structured organization of the alphabet that resembles the structure used by the optimal strategy. Many selected merges remain largely noninterfering, and the induced partition stays close to balanced.

This suggests that, even after the first conflict, BPE may continue to follow a near-optimal structure for some additional range of steps. Once perfect merges are no longer available, the process becomes more complex because previously selected merges begin to interact with one another. Nevertheless, the observed balance of the partition indicates that the algorithm enters this later regime from a highly favorable state. This supports the broader intuition that BPE performs well on random strings

not only because early pair frequencies are uniform, but because its greedy choices preserve enough structure to remain close to the optimal strategy beyond the initial maximum-utility phase.

Overall, these experiments reinforce the main conclusion of the random-string analysis: BPE behaves nearly optimally when merge opportunities are symmetric and noninterfering. They also point to a natural direction for future work, namely extending the analysis beyond the first $n^2/4$ merge steps to understand how long this near-optimal structure persists.

Chapter 6

Conclusion and open questions

In this work, we presented a refined theoretical analysis of Byte Pair Encoding (BPE) by framing it as a constrained optimization problem and studying its approximation behavior relative to an optimal merge strategy. Our results highlight a nuanced picture of BPE’s performance across different input regimes.

First, we established a nearly tight upper bound, showing that BPE cannot guarantee better than a $(\frac{1}{2} + \varepsilon)$ -approximation in the worst case. This demonstrates an inherent limitation of the greedy strategy underlying BPE, even under carefully structured inputs. Complementing this, we proved a constant-factor lower bound of $\frac{1-e^{-2}}{2}$ on a restricted but nontrivial class of inputs, showing that BPE retains meaningful guarantees when interactions between merges are controlled.

Our analysis of uniformly random strings suggests that BPE behaves dramatically better in settings with highly uniform local structure. For the first $\frac{n^2}{4}$ merge steps, it achieves near-optimal utility in expectation. In contrast to worst-case constructions, where local frequency information is highly skewed and can be misleading, the uniform setting ensures that merge opportunities are nearly symmetric and do not interfere significantly. This separation between worst-case and average-case behavior helps explain the strong empirical performance of BPE in practical natural

language processing applications, despite its theoretical limitations. More broadly, this contrast points toward a structural explanation for BPE’s behavior. The algorithm performs well when local merge frequencies are well-aligned with global utility, and degrades when this alignment breaks down. This also highlights why length-4 substrings suffice to realize worst-case behavior. They are the smallest patterns in which merge positions can have asymmetric effects. The interior merges eliminate two future opportunities for relatively large merges, while boundary merges eliminate only one. This asymmetry creates locally attractive but globally harmful choices, exposing the core weakness of greedy selection in BPE.

A useful way to interpret our results is to view the analyzed input distributions as representing two extremes of structural interaction between merge positions. On one end, uniformly random strings exhibit near-perfect symmetry, where all adjacent pairs have approximately equal frequency, and merge decisions do not significantly interfere with one another. In this regime, every position is effectively “neutral,” and BPE’s greedy rule aligns closely with optimal behavior.

On the other end, our length-4 construction illustrates a setting with highly asymmetric interactions between merge positions. In these strings, certain merges, particularly those in the middle of substrings, appear locally optimal but destroy multiple future opportunities, creating what can be viewed as “bad” merge positions. These positions are deceptive, in that they exploit the greedy nature of the algorithm. They have high immediate utility but incur significant opportunity cost by eliminating both adjacent merges. The worst-case behavior of BPE arises precisely when such misleading positions dominate the decision process.

Taken together, these two regimes suggest that the approximation performance of BPE is governed by the density and structure of such “bad” merge positions. The uniform distribution represents a setting where no such positions exist, while the four-character construction maximizes their impact. This perspective provides intuitive

evidence that our upper bound is essentially tight, and suggests that inputs with fewer interfering merge interactions should admit stronger approximation guarantees. In particular, we conjecture that no input can induce more destructive interference than this regime, and that the $\frac{1}{2}$ approximation is tight.

Several directions remain open. A central question is to prove or refute this conjecture, namely whether the $\frac{1}{2}$ upper bound is tight for general inputs. We believe the four-letter word structure to be the worst possible for BPE, and that as words get longer, BPE should only improve, but this remains to be proven. It would be valuable to extend the lower-bound guarantees beyond highly structured inputs, or to characterize broader classes of strings where BPE achieves constant or near-optimal approximations. Our analysis of strings drawn uniformly at random relies on symmetry-based conjectures; formally proving these conjectures, or replacing them with unconditional bounds, is an important next step. Even considering strings drawn from a weighted distribution would better model natural language, and would be a very interesting continuation of this research.

More broadly, our work suggests that the behavior of BPE is closely tied to structural properties of the input distribution. Understanding how non-uniform or linguistically realistic distributions affect its approximation guarantees remains an intriguing direction for future research. Finally, analyzing the behavior of BPE beyond the first $\frac{n^2}{4}$ steps on a uniform string, where interactions between merges become increasingly complex, may yield further insight into how optimal the algorithm truly is.

Overall, this work provides a step toward a more complete theoretical understanding of BPE, bridging the gap between its simplicity as an algorithm and its effectiveness in practice.

Bibliography

- [1] Philip Gage. A new algorithm for data compression. *C Users Journal*, 1994.
- [2] László Kozma and Johannes Voderholzer. Theoretical analysis of byte-pair encoding. *arXiv preprint arXiv:2411.08671*, 2024.
- [3] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of ACL*, 2016.
- [4] Avi Shein and Timofei Asinski. Bpe-proj-2025. <https://github.com/tasinskii/BPE-proj-2025>, 2025. GitHub repository.
- [5] Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Tim Vieira, Mrinmaya Sachan, and Ryan Cotterell. A formal perspective on byte-pair encoding. *arXiv preprint*, 2023.

Appendix A

Analysis of the Random Maximum-Edge Model

We provide the full derivation of the expectation of D^2 under the random maximum-edge model. Our goal is to estimate how the imbalance between the sets V_1 and V_2 evolves as new symbols are assigned during the process.

Unlike Conjectures 1 and 2, where exactly one new symbol is assigned at each step, the random maximum-edge model may assign either one or two new symbols at a time. In particular, selecting an edge from $U \times U$ assigns two previously unassigned symbols simultaneously, while selecting an edge involving exactly one unassigned symbol assigns only one new symbol. Consequently, the total number of process steps is itself random, and it is more natural to analyze the evolution of the imbalance as a function of the number of assigned symbols rather than the number of merge steps.

We therefore approximate the process continuously and derive a differential equation governing the expected imbalance.

Let

$$a = |V_1|, \quad b = |V_2|, \quad r = n - a - b, \quad D = a - b$$

.

Let I_j denote the number of assigned symbols after the j -th step of the process, and let D_j denote the corresponding imbalance.

Conditioned on selecting a maximum-utility pair involving at least one previously unassigned symbol, the possible transitions are

$$D_{j+1} = \begin{cases} D_j, & \text{with probability } \frac{r-1}{n-1}, \\ D_j + 1, & \text{with probability } \frac{b}{n-1}, \\ D_j - 1, & \text{with probability } \frac{a}{n-1}. \end{cases}$$

Therefore,

$$\mathbb{E}[D_{j+1}^2 \mid D_j, I_j = i] = D_j^2 \frac{r-1}{n-1} + (D_j + 1)^2 \frac{b}{n-1} + (D_j - 1)^2 \frac{a}{n-1}.$$

Expanding and simplifying using $D = a - b$, $a + b = i$, gives

$$\mathbb{E}[D_{j+1}^2 \mid D_j, I_j = i] = D_j^2 \left(1 - \frac{2}{n-1}\right) + \frac{i}{n-1}.$$

Hence,

$$\mathbb{E}[D_{j+1}^2 - D_j^2 \mid D_j, I_j = i] = \frac{i - 2D_j^2}{n-1}.$$

The true process is indexed by the step number j , but once we condition on $I_j = i$, the transition probabilities above depend only on the current values of i and D_j , and not explicitly on j . Thus we reparametrize the expected evolution by the number of assigned symbols rather than by the number of process steps.

Define

$$M(i) = \mathbb{E}[D_j^2 \mid I_j = i],$$

so that $M(i)$ represents the expected squared imbalance conditioned on the event that i symbols have been assigned. Replacing D_j^2 by its conditional expectation $M(i)$, the

expected change in M is

$$\Delta M = \frac{i - 2M(i)}{n - 1}.$$

Next, we compute the expected increase in the number of assigned symbols per step, denoted Δi .

A merge from $U \times U$ assigns two new symbols, while the remaining cases assign one:

$$\Delta i = 2 \left(\frac{n - i - 1}{n - 1} \right) + 1 \left(\frac{i}{n - 1} \right).$$

Simplifying,

$$\Delta i = \frac{2(n - i - 1) + i}{n - 1} = \frac{2(n - 1) - i}{n - 1}.$$

Therefore,

$$\frac{\Delta M}{\Delta i} = \frac{i - 2M(i)}{2(n - 1) - i}.$$

For large n , we approximate this by the differential equation

$$\frac{dM}{di} \approx \frac{i - 2M}{2n - i}.$$

To solve this differential equation, we begin by rewriting it as

$$\frac{dM}{di} + \frac{2}{2n - i} M = \frac{i}{2n - i}.$$

Multiplying both sides by the integrating factor gives

$$(2n - i)^{-2} \frac{dM}{di} + \frac{2}{(2n - i)^3} M = \frac{i}{(2n - i)^3}.$$

We can recognize the product rule on the left side and rewrite our equation as

$$\frac{d}{di} (M(i)(2n - i)^{-2}) = \frac{i}{(2n - i)^3}.$$

Integrating from 0 to i and using the initial condition $M(0) = 0$ yields

$$M(i) = (2n - i)^2 \int_0^i \frac{s}{(2n - s)^3} ds.$$

Setting $i = n$, we obtain

$$M(n) = n^2 \int_0^n \frac{s}{(2n - s)^3} ds.$$

We can then perform a u substitution with $u = 2n - s$, and then compute

$$\int_0^n \frac{s}{(2n - s)^3} ds = \int_n^{2n} \frac{2n - u}{u^3} du = \int_n^{2n} \left(\frac{2n}{u^3} - \frac{1}{u^2} \right) du.$$

Finally, we evaluate this on the bounds of n to $2n$

$$= \left[-\frac{n}{u^2} + \frac{1}{u} \right]_{u=n}^{2n} = \frac{1}{4n}.$$

Therefore, after all n symbols have been assigned, our continuous approximation yields

$$M(n) = \frac{n}{4}.$$

Recall that $M(i)$ was defined to represent the expected value of the squared imbalance after i symbols have been assigned, namely

$$M(i) = \mathbb{E}[D^2 \mid I = i].$$

Consequently, evaluating this function at $i = n$ gives the expected final squared imbalance of the process:

$$\mathbb{E}[D^2] = \frac{n}{4}.$$

Finally, since

$$|V_1||V_2| = \frac{n^2 - D^2}{4},$$

we conclude that

$$\mathbb{E}[|V_1||V_2|] = \frac{n^2}{4} - \frac{n}{16}.$$